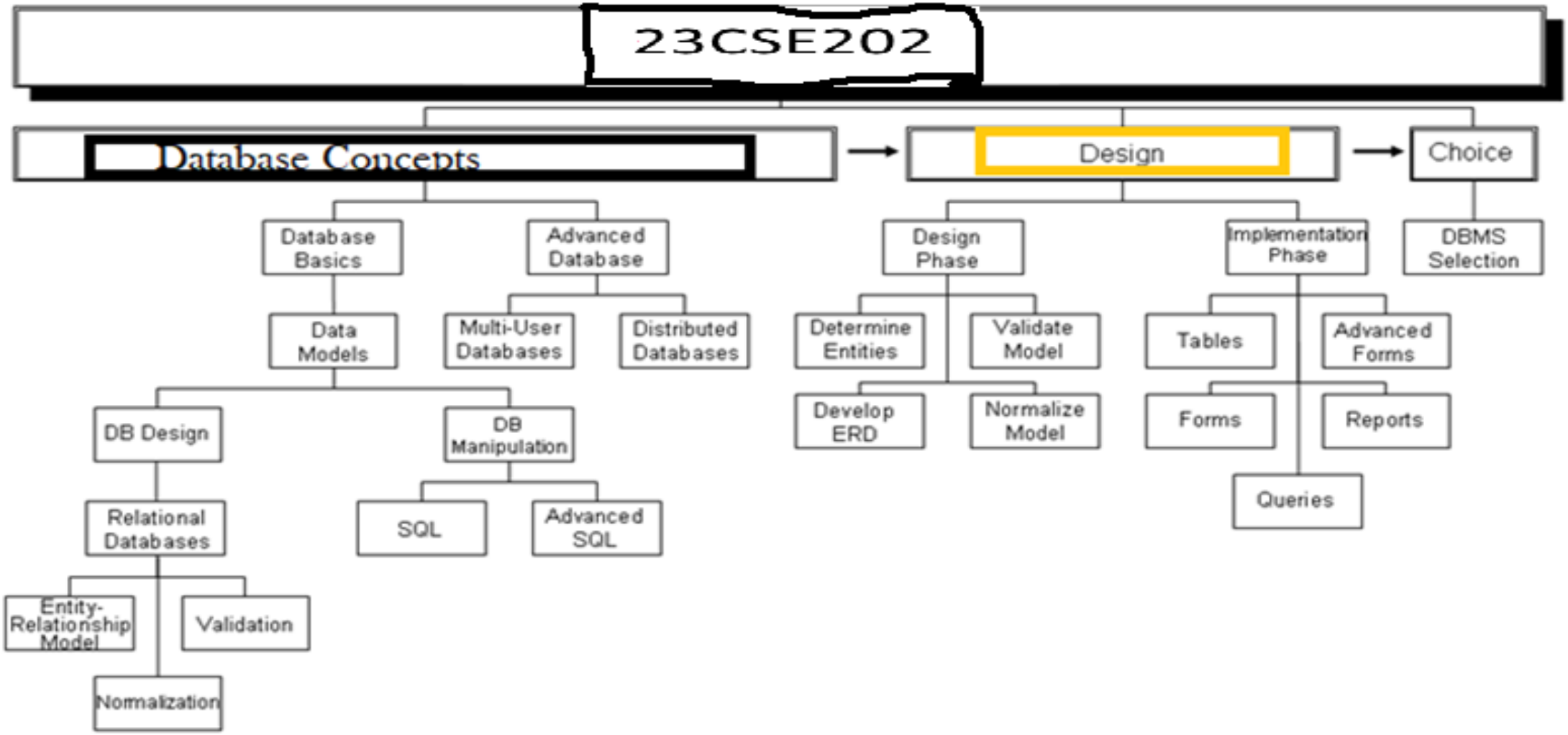
23CSE202-DBMS

SQL BASICS

Overview DBMS



Introduction to SQL



- SQL is an acronym of Structured Query language
- It is standard language developed and used for accessing and modifying relational databases.
- Structured query language (SQL) is a programming language for storing and processing information in a relational database.
- You can use SQL statements to store, update, remove, search, and retrieve information from the database. You can also use SQL to maintain and optimize database performance.

Introduction to SQL



Introduction to SQL

- SQL is being used by many database management systems. Some of them are:

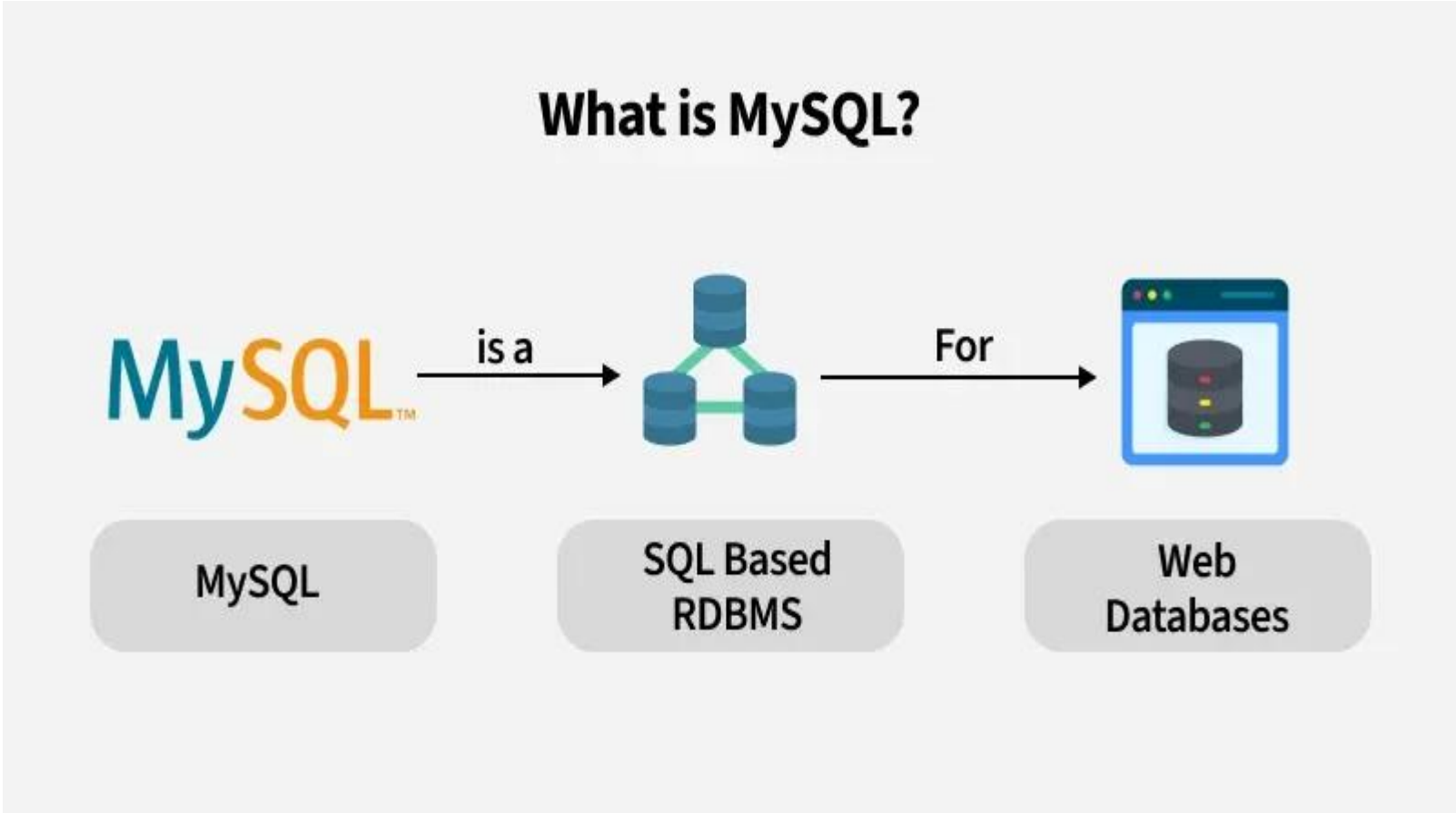
  - MySQL
  - PostgreSQL
  - Oracle
  - SQLite
  - Microsoft SQL Server

MYSQL



- MySQL is a popular open-source relational database management system (RDBMS) that uses SQL (Structured Query Language) to manage and manipulate data stored in tables.
- It is developed by Oracle Corporation.
- It supports multiple platforms like Windows, Linux, and macOS.
- It's widely used for various applications, from simple websites to complex enterprise systems, due to its reliability, performance, and scalability.

MYSQL

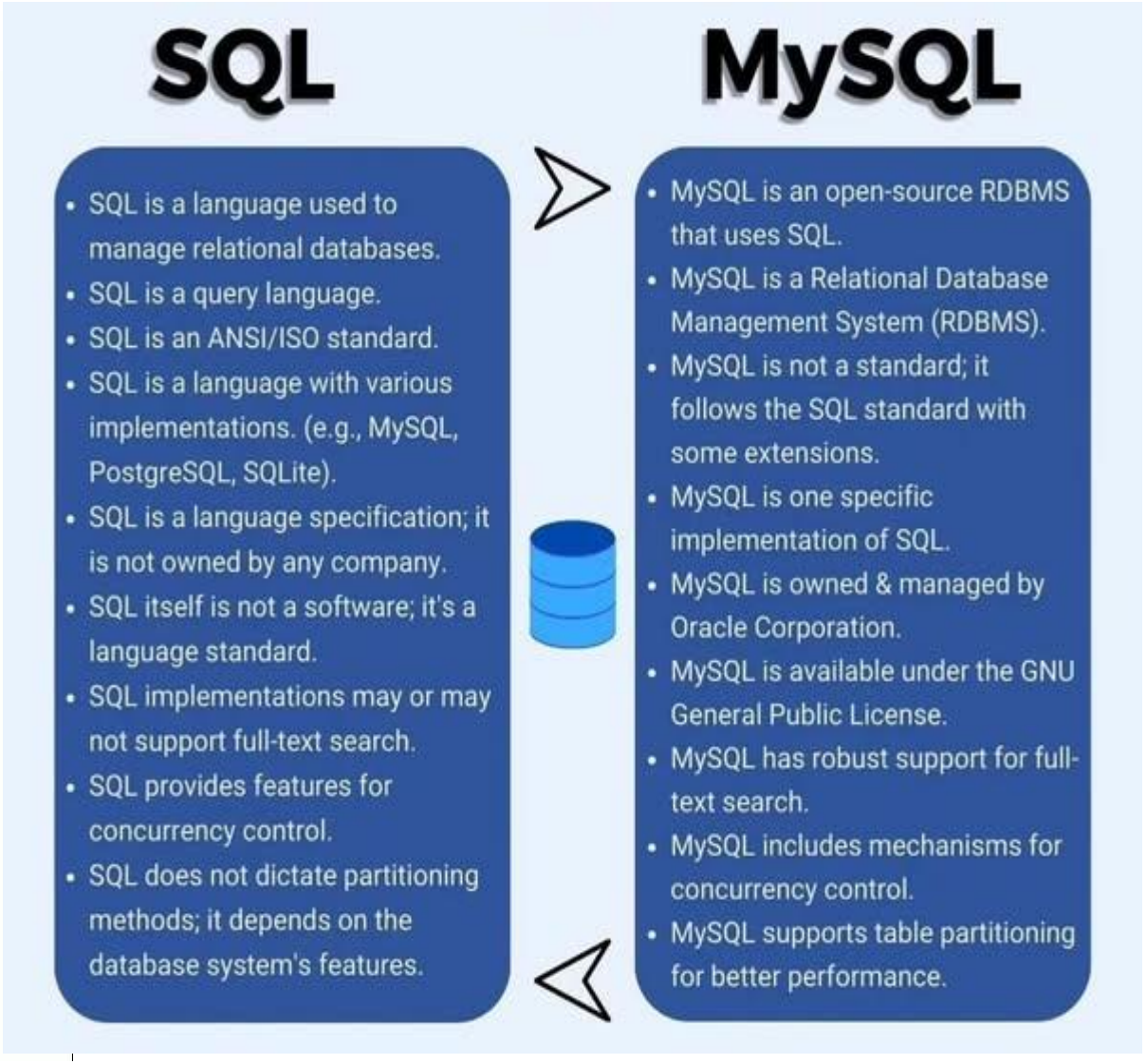

What is MySQL?
MySQL
is a
For
MySQL
SQL Based RDBMS
Web Databases

MYSQL Features

- MySQL is a relational database management system
- MySQL is open-source
- MySQL is free
- MySQL is ideal for both small and large applications
- MySQL is very fast, reliable, scalable, and easy to use
- MySQL is cross-platform
- MySQL is compliant with the ANSI SQL standard
- MySQL was first released in 1995
- MySQL is developed, distributed, and supported by Oracle Corporation

SQL Vs MYSQL

SQL	MySQL
SQL is a language to manage databases.	MySQL is a database software.
SQL is used to query databases.	MySQL stores the data.
SQL is structured query language.	MySQL is RDBMS (Relational Database Management System)
SQL does not provide connectors.	MySQL provide an integrated tool called "MySQL workbench"
SQL codes or commands are used in Oracle, SQL server, PostgreSQL, DB2, MariaDB, MySQL etc.	MySQL uses SQL.



SQL Commands

The standard SQL commands to interact with relational databases are CREATE, SELECT, INSERT, UPDATE, DELETE and DROP.

These commands can be classified into groups based on their nature:

DDL (Data Definition Language)

- To create database and table structure.

DML (Data Manipulation Language)

- Record/rows related operations.

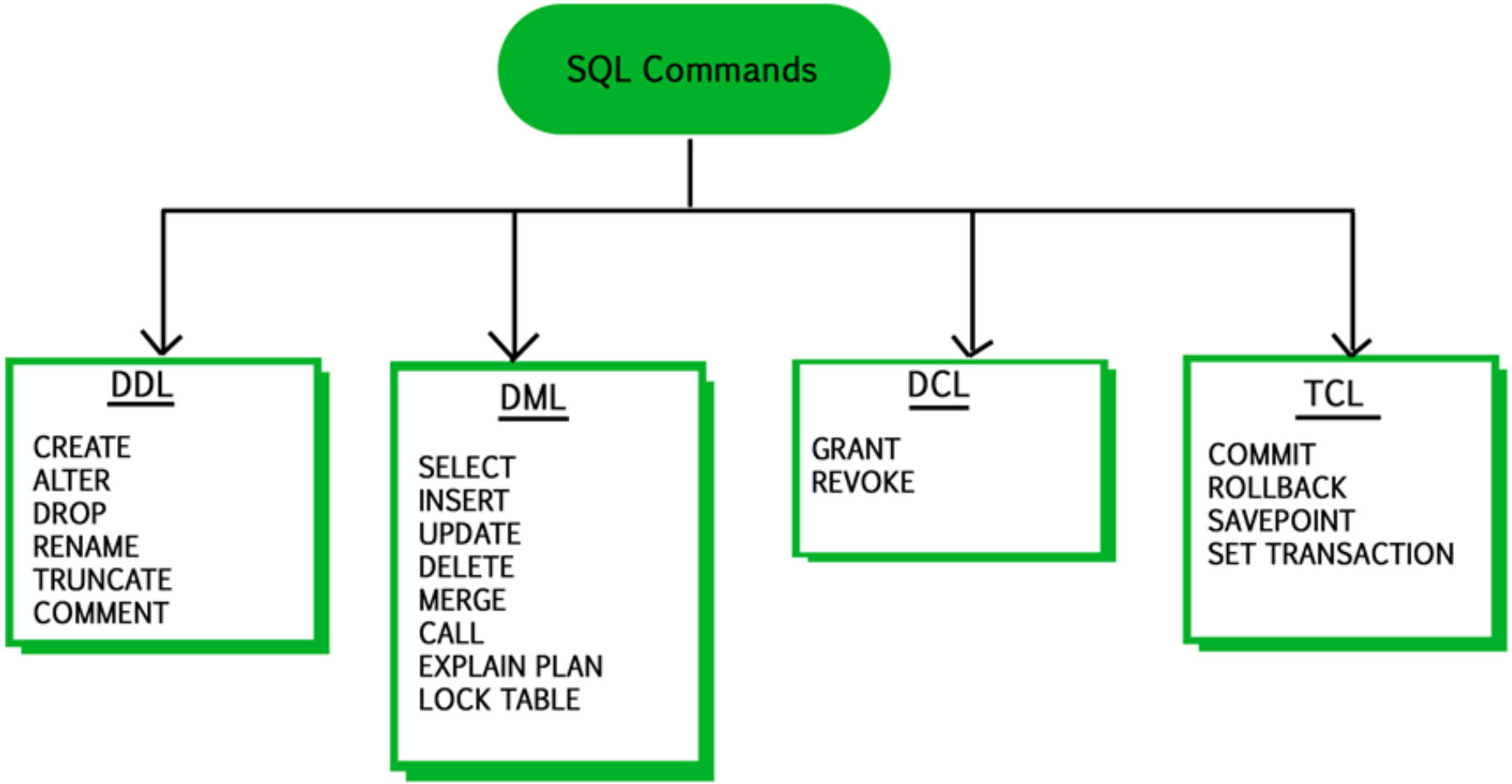
DCL (Data Control Language)

- used to manipulate permissions or access rights to the tables.

TCL (Transactional control Language)

- Used to control the transactions.

SQL Structured Query Language



DDL - Data Definition Language

Command	Description
CREATE	Creates a new table, a view of a table, or other object in database
ALTER	Modifies an existing database object, such as a table.
DROP	Deletes an entire table, a view of a table or other object in the database.

DML - Data Manipulation Language

Command	Description
SELECT	Retrieves certain records from one or more tables
INSERT	Creates a record
UPDATE	Modifies records
DELETE	Deletes records

DCL - Data Control Language

Command	Description
GRANT	Gives a privilege to user
REVOKE	Takes back privileges granted from user

Transaction Control (TCL)

Command	Description
COMMIT	Save work done
ROLLBACK	Restore database to original since the last
SAVEPOINT	Identify a point in a transaction to which you can later roll back

Table or relation

- The data in RDBMS is stored in database objects called **tables or relations.**
- Every table is broken up into smaller entities called fields or attributes.
- The fields in the CUSTOMERS table consist of ID, NAME, STREET and CITY.
- A record, also called a row or tuple of data, is each individual entry that exists in a table.
- Each customer detail is a tuple

customer-id	*customer-name*	*customer-street*	*customer-city*
192-83-7465	Johnson	12 Alma St.	Palo Alto
019-28-3746	Smith	4 North St.	Rye
677-89-9011	Hayes	3 Main St.	Harrison
182-73-6091	Turner	123 Putnam Ave.	Stamford
321-12-3123	Jones	100 Main St.	Harrison
336-66-9999	Lindsay	175 Park Ave.	Pittsfield
019-28-3746	Smith	72 North St.	Rye

(a) The *customer* table

Constraints

Constraints available in SQL:

- **NOT NULL** : Ensures that a column cannot have NULL value.
- **DEFAULT** : Provides a default value for a column when none is specified.
- **UNIQUE** : Ensures that all values in a column are different.
- **PRIMARY Key**: Uniquely identified each rows/records in a database table.
- **FOREIGN Key**: Uniquely identified a rows/records in any another database table.
- **CHECK** : The CHECK constraint ensures that all values in a column satisfy certain conditions.

SQL Data types

- SQL data types are broadly categorized into several groups:

  - Numeric Data Types
  - Character and String Data Types
  - Date and Time Data Types
  - Binary Data Types
  - Boolean Data Types
  - Special Data Types

SQL Server String Data Type

Data Type	Description	Size
CHAR(size)	A character string data type with a fixed width.	8000 characters
VARCHAR(size)	A character string data type with configurable width.	8000 characters
VARCHAR(max)	A Character string data type with configurable width.	1,073,741,824 characters
TEXT	A character string data type with configurable width.	2 GB of text data

SQL Server Numeric Data Types

Data Type	Description	Size
BIT	It is an integer that can have a value of 0 (zero), 1 (one), or null.	
TINYBIT	A set of whole numbers ranging from 0 to 255.	1 byte
SMALLINT	A set of whole numbers ranging from -32,768 to 32,767.	2 bytes
INT	A set of whole numbers ranging from -2,147,483,648 to 2,147,483,647.	4 bytes
BIGINT	A set of whole numbers ranging from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807.	8 bytes
FLOAT(n)	Data with floating precision ranging from -1.79E+308 to 1.79E+308. The n option specifies whether the field should carry four or eight bytes. n has a default value of 53.	4 or 8 bytes
REAL	Data with a floating precision ranging from -3.40E+38 to 3.40E+38.	4 bytes
SMALLMONEY	Specifies monetary data ranging from -214,748.3648 to 214,748.3647.	4 bytes
MONEY	Specifies monetary data ranging from -922,337,233,685,477.5808 to 922,337,203,685,477.5807.	8 bytes
DECIMAL(p, s)	A set of Numbers with fixed precision and scale p specifies the maximum number of decimal digits that can be saved. s specifies the number of decimal digits stored to the right of the decimal point. When using the highest precision, acceptable numbers range from - 1038 +1 to 103	5-17 bytes
DEC(p, s)	As same as DECIMAL(p, s).	5-17 bytes
NUMERIC(p,s)	As same as DECIMAL(p, s).	5-17 bytes

SQL Server Date and Time Data Type

Data Type	Description	Format
DATETIME	A combination of date and time. With an accuracy of 3.33 milliseconds, it ranges from January 1, 1753 to December 31, 9999.	YYYY-MM-DD hh:mm.mmm
DATETIME2	A combination of date and time. With an accuracy of 100 nanoseconds, it supports a range of January 1, 0001 to December 31, 9999.	YYYY-MM-DD hh:mm.fractional seconds
DATE	Denotes a date. It is compatible with dates ranging from January 1, 0001 through December 31, 9999.	YYYY-MM-DD
TIME	Denotes a time. Time was only stored to an accuracy of 100 nanoseconds. The range varies from '00:00:00.0000000' to '23:59:59.9999999'.	YYYY-MM-DD hh:mm.nnnnnnn
DATETIMEOFFSET	A combination of date and time that incorporates time zone awareness based on UTC. With a precision of 100 nanoseconds, it supports a range of January 1, 0001 to December 31, 9999.	YYYY-MM-DD hh:mm.nnnnnnn [{+,-}hh]
SMALLDATETIME	A combination of time and date. It has a range of January 1, 1900 to June 6, 2079 with a precision of 1 minute.	YYYY-MM-DD hh:mm
TIMESTAMP	When a new row is added or changed, it stores a unique number. The timestamp value is based on an internal clock and does not correlate to real-time. Each table can only have one timestamp variable.	

Binary Data Types in SQL

Data Type	Description	Max Length
Binary	Fixed-length binary data.	8000 bytes
VarBinary	Variable-length binary data.	8000 bytes
Image	Stores binary data as images.	2,147,483,647 bytes

Boolean Data Type in SQL

Data Type	Description
BOOLEAN	Stores a logical value (TRUE/FALSE).

Types of SQL Operators

- SQL operators can be categorized based on the type of operation they perform. Here are the primary types of SQL operators:

  - Arithmetic Operator
  - Comparison Operator
  - Logical Operator
  - Bitwise Operators
  - Compound Operators
  - Special Operators

SQL Arithmetic Operators

Operator	Description
+	The addition is used to perform an addition operation on the data values.
-	This operator is used for the subtraction of the data values.
/	This operator works with the 'ALL' keyword and it calculates division operations.
*	This operator is used for multiplying data values.
%	Modulus is used to get the remainder when data is divided by another.

SQL Comparison Operators

Operator	Description
=	Equal to.
>	*Greater than.*
<	Less than.
>=	Greater than equal to.
<=	Less than equal to.
<>	Not equal to.

SQL Logical Operators

Operator	Description
AND	Logical AND compares two Booleans as expressions and returns true when both expressions are true.
OR	Logical OR compares two Booleans as expressions and returns true when one of the expressions is true.
NOT	Not takes a single Boolean as an argument and change its value from false to true or from true to false.

SQL Bitwise Operators

Operator	Description
&	Bitwise AND operator
|	Bitwise OR operator
^	Bitwise XOR (exclusive OR) operator
~	Bitwise NOT (complement) operator
<<	Left shift operator
>>	Right shift operator

SQL Special Operators

Operators	Description
ALL	ALL is used to select all records of a SELECT STATEMENT. It compares a value to every value in a list of results from a query. The ALL must be preceded by the comparison operators and evaluated to TRUE if the query returns no rows.
ANY	ANY compares a value to each value in a list of results from a query and evaluates to true if the result of an inner query contains at least one row.
BETWEEN	The SQL BETWEEN operator tests an expression against a range. The range consists of a beginning, followed by an AND keyword and an end expression.
IN	The IN operator checks a value within a set of values separated by commas and retrieves the rows from the table that match.
EXISTS	The EXISTS checks the existence of a result of a subquery. The EXISTS subquery tests whether a subquery fetches at least one row. When no data is returned then this operator returns 'FALSE'.
SOME	SOME operator evaluates the condition between the outer and inner tables and evaluates to true if the final result returns any one row. If not, then it evaluates to false.
UNIQUE	The UNIQUE operator searches every unique row of a specified table.

CREATE TABLE

Creates a table with given table name

CREATE TABLE table_name (column1 datatype, column2 datatype,....);

Eg:

```
CREATE TABLE employee(   empid int,   empname varchar(20), salary float);
```

INSERT VALUES IN TO TABLE

- Used to populate the table

```
INSERT INTO table_name (column1, column2, column3, ...)
VALUES (value1, value2, value3, ...);
```

Eg:

```
INSERT INTO employee (empid, empname,salary) VALUES(200,'gokul',2300);
```

(OR)

```
INSERT INTO employee VALUES(200,'gokul',2300);
```

SELECT-To Retrieve values

- A typical SQL select query has the form:

  select attr1,attr2..attrn from r1, r2, ..., rm where condition;

The result of an SQL query is a relation

NOTE: SQL names are case insensitive (i.e., you may use upper- or lower-case letters.)

E.g. Name ≡ NAME ≡ name

SELECT

An asterisk in the select clause denotes “**all attributes**”

- **select * from *table_name;***

- ***Eg:*** select * from *employee;*

The SELECT statement is used to select certain data from a database

- **Eg:**
- select empid from employee;
  select empid,salary from employee;

Select-Where

Display all columns with condition from the table

- **select * from table_name where column_name=value;**

- Eg:
- Select *from employee where empid=200;

Display particular columns with condition from the table

- select empname,salary from employee where empid=200;

SELECT-Distinct

- SQL allows **duplicates** in relations as well as in query results.

- To force the **elimination of duplicates**, insert the keyword **distinct** after select**.**
- **Find the salary of all employees from employee table, and remove duplicates**

select distinct *salary* from *employee;*

- The keyword **all** specifies that duplicates not be removed.

select all *salary* **from** *employee ;*

SELECT

- The **select** clause can contain arithmetic expressions involving the operation, +, –, *, and /, and operating on constants or attributes of tuples.

- The query:

  select *empid, empname, salary/12* **from** *employee;*

  would return a relation that is the same as the *employee* relation, except that the value of the attribute *salary* is divided by 12.

References

- https://www.scaler.com/topics/sql/sql-data-types/
- https://www.geeksforgeeks.org/sql-operators/